Ce alg se obține dacă avem $h(n) = 0$ la algoritmul A^*?

A^* - algoritm euristic ce folosește două funcții de costuri:

$h(n)$ = costul estimat de la nodul curent la nodul finish

$g(n)$ = distanța parcursă de la nodul inițial până la nodul n

La fiecare iterație se va alege din multimea OPEN(c) cel mai mic nod având euristică: $f(n) = h(n) + g(n)$.

Dacă $h(n) = 0 \Rightarrow$ Se va alege mereu $f(n) = g(n)$ = distanta parcursă minimă ceea ce duce la algoritmul lui Dijkstra

Precizați 2 euristici pentru problema aranjării a 8 pătrate. Ce putem spune despre aceste euristici dpdv A^*?

Problema aranjării a 8 pătrate constă în mutarea locului "liber" pana se ajunge la forma finală. Distingem 2 euristici

h_1 = nr pătrate a căror poziție curentă diferă de cea finală

$= \sum_{\text{pepiese}} \delta(i)$, $\delta(i) = \begin{cases} 0 & \text{dacă } i \text{ e poziția finală} \\ 1 & \text{altfel} \end{cases}$

h_2 = distanța Manhattaan = suma distanțelor pe verticală și pe orizontală între pozițiile curente și pozițiile finale.

$= \sum_{\text{pepiese}} (dist_h(p) + dist_o(p))$

Algoritmul A^* ghidat printr-o euristică admisibilă descoperă soluția optimă.

O funcție euristică h este admisibilă dacă $\forall$ nod e spațiul stărilor $h(n) \leq h^*(n)$ unde $h(n)$ = costul estimat al drumului de la n la soluție, iar $h^*(n)$ = costul exact al drumului

Demonstrăm că h_1, h_2 sunt admisibile, deoarece subestimează nr real de mutări

h_1: mută o piesă dacă $\exists$ a modif și tb să facem a sg mutare (sigur vor fi > mutări)

h_2: spațiul va fi influențat și de spațiul înapoi $\Rightarrow$ subestimează

h_2 este un algoritm dominant peste h_1 deoarece expandează toate nodurile din h_1, cu mai multe noduri $\Rightarrow$ h_2 dominant

Scanned with CamScanner

Schema MINIMAX pentru un joc de SAH

Pentru a soluționa un joc de sah folosind algoritmul MIN MAX as folosi o funcție euristică pentru a calcula estimat șansele de câștig pentru o poziție dată.

Pentru fiecare tip de piesa vom asigna o valoare:

Regina = 5p
Pion = 1p
Cal = 3p

iar funcția de evaluare a poziției va fi suma pieselor proprii - piese adversare

$f = \Sigma$ piese_proprii $- \Sigma$ piese_adversare

Enumerați utilizările timpilor de debut și finish în adâncime a unui graf. parcurgerea în

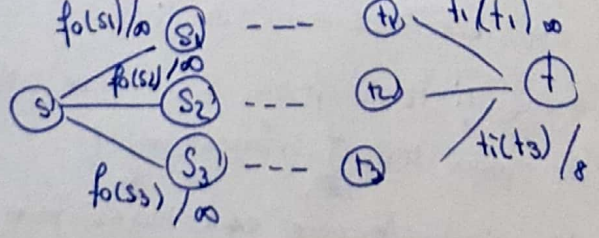
1) Topo - Sort (sortare topologică) ✓
2) Găsirea componentelor conexe ✓
3) Descoperirea muchiilor critice și a punctelor de articulație (Tarjan) ✓
4) Detectare cicluri ✓
5) Tarjan SCC ✓

Cum se poate transforma o problemă de flux maxim într-o rețea cu mai multe noduri și mai multe destinații într-o problemă de det a fluxului maxim într-o rețea cu o sursă și o destinație.

Fie surse multiple $\{s_1, s_2 \dots s_m\}$ și destinații multiple $\{t_1, t_2 \dots t_m\}$

Pentru a reduce problema la o problemă de flux maxim cu o sursă și o destinație se adaugă o sursă unică cu arce de capacitate ∞ spre sursele $s_1, s_2 \dots s_m$ și flux egal cu fluxul generat de $s_1 \dots s_m$.

Se adaugă o destinație unică t și arce de capacitate infinită între $t_1 \dots t_m$ și flux egal cu ce intră în destinațiile respective



Scanned with CamScanner

GREEDY

- problema are proprietate de substructură optimă
  (soluția problemei conține soluțiile subproblemei)
- problema are proprietatea alegerii locale
  (alegând soluția optimă local se ajunge la soluția optimă global)
- Numim E o mulțime finită și $I \subset P(E)$ a.î $\emptyset \in I$, $\forall x \subset Y, Y \in I \Rightarrow x \in I$. Spunem că (E,I) este o mulțime accesibilă, iar submulțimile din I se numesc independente
- Un sistem accesibil este un matroid dacă satisface proprietatea de interschimbare
$\forall X, Y \in I \quad |X| < |Y| \Rightarrow \exists e \in Y / X$ a.î $X \cup \{e\} \in I$ Dacă (Ei) este un matroid, atunci $\exists$ alg Greedy care rezolvă problemă de optimizare

↳ de notat

Scanned with CamScanner

Comparați algoritmii Las Vegas și Monte Carlo

Las Vegas și Monte Carlo sunt algoritmi aleatori.
Las Vegas determină soluția optimă, renunțându-se la optimalitate, soluția suboptimală are o marjă de eroare garantată prin calcul probabilistic.
Monte Carlo determină soluția sigură, găsește o soluție ce se apropie cu o exactitate probab. de soluția exactă.
Las Vegas găsește soluția corectă a problemei, însă timpul de rezolvare nu poate fi determinat cu exactitate. Cu cât crește timpul de rulare, cu atât crește probabilitatea mai mult.
Monte Carlo găsesc o soluție a problemei care nu e garantat corectă. Probabilitatea crește cu timpul de rezolvare

Ce se poate modifica la Dijkstra pt DAG?
Se folosește topo-sort $O(V+E) < O(E+V\log V)$ - Dijkstra

Ce înseamnă o euristică admisibilă A*) Ce s-ar întâmpla dacă nu se respectă

Algoritmul A* ghidat de o euristică admisibilă soluționează optim o problemă.
O euristică este admisibilă dacă pt ∀ nod ∈ spațiu stărilor $h(m) \leq h'(m)$, unde $h(m)$ = distanța estimată de la nodul m la sursă, $h'(m)$ = costul exact al drumului. Dacă A* nu are euristică admisibilă, nu găsește sol exactă.

Un robot de căutare 'mandarină'. Descrieți 2 situații în care algo. e ineficient sau chiar nu găsește soluția optimă.

Algoritmul descris este un algoritm euristic informat inevocabil, deoarece pentru fiecare nod din spațiul stărilor se calculează o valoare f(nod), spre exemplu distanța spre cel mai înalt vârf și de fiecare dată se alege din mulțimea de posibilități nodul cu valoarea cea mai mare. Un caz în care alg. nu ar merge ar fi, de exemplu, ca drumul optim să nu fie format din cele mai înalte vârfuri.

Scanned with CamScanner

La parcurgerea în adâncime în cazul cel mai superficial se fac 2 iterații. Ce se poate spune despre acest graf?

Pentru un graf $G(V,E)$, $BFS(G)$ sparge graful într-o pădure de arbori $Arb(G) = \{Arb(u) \mid p(u) = null\}$. Astfel, la 2 iterații în cazul cel mai superficial se vor identifica 2 păduri de arbori.

Teorema parantezelor

$\forall u,v \in V$ avem $i(u) \cap i(v) = \emptyset$ sau $i(u) \subset i(v)$ sau $i(v) \subset i(u)$

Se demonstrează prin tratarea tuturor variantelor, iar i este intervalul de prelucrare al unui nod i $(d(i), f(i))$

a) $i(u) < i(v)$

$d(u) < f(u) < d(v) < f(v)$

$\Rightarrow$ v e alb în timpul prelucrării lui u

b) $i(v) \subset i(u)$ $d(u) < d(v) < f(v) < f(u)$

$\Rightarrow$ v e descoperit în timpul prelucrării lui u

c) $i(v) < i(u)$

$d(v) < f(v) < d(u) < f(u)$

$\Rightarrow$ u e alb în t.p prelucrării lui v

d) $i(u) \subset i(v)$ - analog b

Teorema drumurilor albe

Fie $G=(V,E)$, v este descendent al lui u în $Arb(u) \Leftrightarrow$ la momentul $d(u)$ $\exists$ $u \dots v$ cale doar de noduri albe.

1) $v \in Arb(u) \Rightarrow \exists$ cale unică $v \dots \alpha \dots u$ (prin pointerii de părinți)

Fie z un nod oarecare $\in \alpha \Rightarrow d(u) < d(z) < f(z) < f(u) \Rightarrow c(z) =$ alb

$\Rightarrow z$ - nod oarecare $\Rightarrow \exists$ drum noduri albe

2) $\exists$ cale unică $v \dots \alpha \dots u \Rightarrow v \in Arb(u)$

noduri albe

Fie $u\, v_0 v_1 \dots v_p\, v$ - calea de noduri albe. Demonstrăm prin inducție după i că $v_i \in Arb(u)$

i caz bază: $d(u) < d(v_1) < f(u)$

i: Considerăm v_i descendent al u la pas $m \Rightarrow$ demonstrez v_{i+1}

$\Rightarrow d(u) < d(v_i) < f(v_i) < f(u)$. v_{i+1} alb la $d(u)$ și e succesor

$\Rightarrow d(u) < d(v_i) < d(v_{i+1}) < f(v_i) < f(u) \Rightarrow$ adv

Scanned with CamScanner

Taietura minimală - Flux maxim

Fie un graf $G(VE)$ rețea de flux. Fie f o funcție de flux în G astfel încât $|f|$ este flux maxim în G dacă rețeaua reziduală G_f nu are căi reziduale și $\exists$ o tăietură (ST) astfel încât $|f| = c(S,T)$

Proprietăți flux

1) respectarea capacității $f(u,v) \leq c(u,v)$
2) simetrie $f(u,v) = -f(v,u)$
3) conservare $\sum f(u,v) = 0 \quad \forall u,v \neq ST$

Ford Fulkerson
$O(f_{max} E)$
Edmonds Karp
$O(E^2 V)$

Operații flux:

1) $f(x,y) = -f(y,x)$
2) $f(x \setminus y, z) = f(x,z) - f(y,z)$
3) $f(x \cup y, z) = f(x,z) + f(y,z)$
4) $f(s,v) = f(v,T)$

Tăieturi în rețele de flux

O tăietură (ST) a unei rețele de flux G este partiționarea mulțimii V în 2 mulțimi disjuncte $S,T = V - S$ astfel încât $s \in S$ $t \in T$

$$f(S,T) = \sum_{x \in S} \sum_{y \in T} f(x,y) = \text{flux prin tăietură}$$

$$f(T,S) = \sum_{y \in T} \sum_{x \in S} f(y,s) = \text{capacitatea tăieturii}$$

Flux tăietură = flux rețea Flux maxim rețea c limitat de capacitatea tăieturii

Scanned with CamScanner

3 **Johnson + complexitate**

Johnson este o metodă de calcul a drumurilor de cost minim într-un graf ce poate conține și muchii de cost negativ. Johnson se folosește mai des pe grafuri rare și este bazat pe algoritmii Dijkstra și Bellman Ford

Complexitate: $(V^2 \log V + E V)$ - mai bună decât Floyd Warshall pe grafuri rare

Ideea ce stă la bază e că

1) dacă avem doar arce de cost pozitiv, aplicăm alg lui Dijkstra → $O(V^2 \log V)$
2) dacă ∃ muchii de cost negativ se adaugă un nod imaginar s ce se unește ~~s~~ prin toate nodurile cu cost 0. Se calculează prin Bellman - Ford distanțele de la s până la noduri. Apoi, se calculează o pondere $w_1(x,y) = w(x,y) + h(x) - h(y)$ și se aplică alg lui Dijkstra, apoi din fiecare distanță calculată $d(x,y) = w_1(x,y) + h(y) - h(x)$
→ $O(V^2 \log V + EV)$

4. Ce este o **tăietură alfa-beta**?

În metoda **MINMAX** există 2 jucători, MIN și MAX care mută pe rând și folosesc principiile: MAX - urmează să își maximizeze câștigul
MIN - urmează să minimizeze pierderea

Idee: Se construiește un arbore AND-OR în care nivelurile pare sunt ale lui MAX iar cele impare ale lui MIN. Frunzele reprezintă câștigul / pierderea rundei, iar frunzele sunt mutările propriu-zise. În cadrul MINMAX se generează arborele din spațiul stărilor, se evaluează frunzele și li se atribuie valori, urmând ca rezultatele să se propage de la frunze spre rădăcină. La fiecare nivel, MIN alege valoarea cea mai mică dintre copii, iar MAX valoarea cea mai mare.
Însă, pentru eficiență nu se poate construi întreg arborele ⇒ ∃ metode de eficientizare

1) folosirea unor euristici
2) oprirea căutărilor inutile < static / dinamic

Tăierea α-β încearcă să limiteze spațiul de căutare prin eliminarea variantelor ce nu ar cum să fie alese

Se taie o ramura dacă:

α = max valori pt un nod MAX
β = min valori pt un nod MIN

1) Nivel MAX: $\beta \leq \alpha$
2) Nivel MIN: $\alpha \geq \beta$

Tăierea α-β reduce complexitatea MINMAX de la $O(\text{ramificații}^{\text{nivele}})$ => $O(\text{ramificații}^{\text{nivele}})$

Scanned with CamScanner

1) 4 modalități de implementare backtracking.

1. Dandu-se o matrice A cu n linii n coloane, modalitate ef. de a calcula A^m.

Putem folosi exponentiere rapidă, tehnică de Dei prin care dacă avem de calculat A^m vom urma pasii:

$$A^m = \begin{cases} (A^{m/2})^2 & \text{dacă } m \text{ e par} \\ A^{m-1} \cdot A & \text{dacă } m \text{ e impar} \\ 1 & \text{dacă } m \text{ e } 0 \end{cases}$$

Astfel, rezolvăm în $\Theta(\log N)$ dar cum fiecare înmulțire de matrice are $N^3 \Rightarrow N^3 \log N$

2 Comparație BFS Dijkstra

BFS - calculează drumul minim de la un nod la restul nodurilor dun graf pentru un graf neponderat (≠ funcție de cost) (drumuri pct - multipunct ~~tot~~

- ca structuri de date folosește coadă pt politica FIFO
- complexitate $O(N+M)$ N = nr noduri M = nr muchii

Dijkstra - calculează drumuri minime de la un nod la restul nodurilor din tr. un graf cu costuri strict pozitive

- ca structuri de date poate folosi coadă de priorități pentru a extrage mereu distanța minimă
- Coada de priorități poate fi implementată folosind:
  1) heap binar cu constrangerea ~~a ≥ nod~~ u ≥ copii sai unde "≥" este relația de ordine
     $O(E \log V)$ - pt grafuri cu nr mic de muchii comparativ cu
  2) vectori - $O(V^2)$ - metodă bună pt grafuri dense
  3) heap fibonacci (cheie părinte ≤ cheie ↓ copil, mai multi arb
     - după teoretic $O(V \log V + E)$

Scanned with CamScanner

1) 4 modalități de îmbunătățire **back tracking**.

1. Alegerea variabilelor în altă ordine, eventual după euristici
2. Îmbunătățirea reveniriilor din recursivitate (look backward)

   Îmbunătățirea navansurilor din recursivitate (look forward)
3. Evitarea redundanțelor în spațiul de căutare

   (=> evitarea repetării unei căutări care știm că va duce la răspuns greșit (=> tăierea în subarbore
4. Prelucrarea restricțiilor

RESTRICȚII

Definim o restricție c ca fiind o relație între una sau mai multe variabile $v_1, v_2, \dots v_m$, unde fiecare variabilă are domeniul său D_i

Restricțiile pot fi
- binare (specifică 2 moduri)
- unare (specifică domeniul unei variabile)
- mare (relații între N moduri)

un tuplu $(x_1 .. x_m)$ satisface restricția dacă $(x_1, x_2 \dots x_m) \in C(v_1, v_2 .. v_m)$

O restricție este o submulțime a produsului cartezian $D_1 \times D_2 \times .. \times D_m$

Definim o problemă de satisfacere a restricțiilor **PSR** ca fiind un triplet $\langle V D C \rangle$ format din V = multimea modurilor, D = multimea domeniilor lor iar C multimea restricțiilor. O soluție PSR este un tuplu ce conține toate variabilele din V, ce îndeplinesc toate restricțiile din C.

Alg de rezolvare PSR are la bază ideea de propagare a restricțiilor și filtrare a multimii de valori până când sunt îndeplinite toate restricțiile sau multimea de valori e vidă.

PSR binar are doar restricții unare și binare.

RSR poate fi reprezentat prin arbore (graf

Definim:

$a = \max\{|D_i|\}$

P_i = predicat unari

Q_{ij} = predicat binari ij

```
NC-1(m) {
    for (x : D(m))
        if (!Q(x))
            remove x, Di;
}
```

Alg
- rezolvă PSR binar
- variabilele au domenii finite D_i
- procesul de propagare se oprește când --
- se filtrează multimi de valori

Scanned with CamScanner

REVICE (i,j) {

for (x : Di)

for (y : Dj)

if ($\nexists$ y a.î Q(x,y))

remove x

return true

return false

AC-1 - se aplică REVICE până nu se mai realizează nicio schimbare

- $O(ma \cdot a^2 \cdot 2n)$

AC-1() {

NC-1();

$Q = \{(i,j) \mid (i,j) \in E(G)\}$

do {

change = false;

for (i,j) ∈ Q

change = change || REVICE (i,j);

} while (change == false);

$O(a^2 \times ma \times 2n)$

- complexitate REVICE
- se elimină câte un nod
- REVICE șterge câte o valoare din restricții binare

- $O(a^3 n)$

- se elimină câte o restricție pe rând
- dacă apare o modificare a restricții se adaugă în coadă restricțiile care pointează către modul de plecare.

AC-3 () {

NC-1();

$Q = \{(i,j) \mid (i,j) \in E(G)\}$

while (Q.empty())

select & delete o muchie (i,j)

if (REVICE (i,j))

$Q = Q \cup \{(k,i) \mid (i,k) \in Edges\}$

}

Scanned with CamScanner

REVICE (i,j) {

for (x : Di)

for (y : Dj)

if (∄ y a.î Q(x,y))

remove x

return true

return false

AC-1 - se aplică REVICE până nu se mai realizează nicio schimbare

- $O(ma \cdot a^2 \cdot 2n)$

$O(a^2 \times ma \times 2n)$

↑ complexitate REVICE

↑ se elimină pe rând câte un mod

REVICE șterge câte o valoare din restricții binare

AC-1 () {

NC-1();

Q = {(ij) | (ij) ∈ E(G)}

do {

change = false;

for (ij) ∈ Q

change = change || REVICE (ij);

} while (change == false);

- $O(a^3 n)$

- se elimină câte o restricție pe rând

- dacă apare o modif (a restricții) se adaugă în coadă restricțiile care pointează către modul de plecare.

AC-3 () {

NC-1();

Q = {(ij) | (ij) ∈ E(G)}

while (Q.empty())

select & delete o muchie (ij)

if (REVICE (ij))

Q = Q ∪ {(ki) | (i,k) ∈ Edges}

}

Scanned with CamScanner